parse_fastq.py 사용 설명서

생물정보학실험 lab11 · 파일 위치: lab11/src/parse_fastq.py · 작성일 2026-11-10

1. 개요

FASTQ 파일을 읽어 각 read 를 (이름, 서열, 품질) 튜플로 만든 뒤, 전체 read 개수와 첫 번째 read 의 이름·서열·길이를 화면에 출력하는 스크립트다. 파이썬 표준 라이브러리(`sys`)만 사용하므로 별도 설치 없이 기본 python3 로 실행할 수 있다.

2. 실행 방법

인자로 FASTQ 파일 경로 하나를 받는다. 저장소 루트(`$MYHOME`)에서 실행하는 예:

```
python3 lab11/src/parse_fastq.py lab11/data/SRR1553607_1.fastq
python3 lab11/src/parse_fastq.py lab11/data/SRR1553607_2.fastq
```

lab11 폴더 안에서 실행하는 경우:

```
cd $MYHOME/lab11
python3 src/parse_fastq.py data/SRR1553607_1.fastq
```

결과를 파일로 남기려면 리다이렉션을 쓴다:

```
python3 src/parse_fastq.py data/SRR1553607_1.fastq > doc/parse_fastq_R1.txt
```

3. 실행 결과 예시

`data/SRR1553607_1.fastq` (약 56 MB, 813,780 줄)에 대해 실행한 실제 출력:

```
read 개수: 203445
첫 read 이름: SRR1553607.1 1 length=101
첫 read 서열: GTTAGCGTTGTTGATCGCGACGCAACAACTGGTAAAGAATCTGGAAGAAGGATATCAGTTCAAACG
            CTCAAGCGAGATGATGGATATTTTTGAACGACTCA
첫 read 길이: 101
```

※ 서열은 실제로 한 줄로 출력되며, 여기서는 지면 때문에 줄을 나눴다. R2 파일(`SRR1553607_2.fastq`)도 read 개수는 203,445 개로 같고, 첫 read 이름도 같지만 서열은 다르다 (paired-end 의 짝 read).

4. 입력 형식: FASTQ

FASTQ 는 read 하나를 정확히 **4줄**로 기록한다. 이 스크립트는 이 4줄 규칙을 전제로 동작한다.

```
@SRR1553607.1 1 length=101          ← 1줄: '@' + read 이름
GTTAGCGTTGTTGATCGCGACGCAACAAC...      ← 2줄: 염기 서열
+SRR1553607.1 1 length=101          ← 3줄: '+' (구분선, 이름 반복은 선택)
BB@FFFFFHHHHHJJJJJJJJJJJJJJJJ...      ← 4줄: 품질 문자열 (서열과 길이가 같음)
```

줄	내용	스크립트에서의 처리
1	@ + read 이름	앞의 '@' 를 떼고 줄 전체를 이름으로 저장
2	서열	그대로 저장 (대소문자 변환 없음)
3	+ 구분선	읽지 않고 건너뜀
4	품질(Phred+33 ASCII)	문자열 그대로 저장 (점수로 변환하지 않음)

5. 코드 설명

```
1  import sys
2
3  def read_fastq(filename):
4      records = []
5      with open(filename) as f:
6          lines = f.readlines()
7
8      for i in range(0, len(lines), 4):
9          name = lines[i].rstrip()[1:]
10         seq = lines[i + 1].rstrip()
11         qual = lines[i + 3].rstrip()
12         records.append((name, seq, qual))
13
14     return records
15
16 reads = read_fastq(sys.argv[1])
17 print("read 개수:", len(reads))
18 print("첫 read 이름:", reads[0][0])
19 print("첫 read 서열:", reads[0][1])
20 print("첫 read 길이:", len(reads[0][1]))
```

줄	설명
1	`sys` 모듈을 불러온다. 명령줄 인자(`sys.argv`)를 쓰기 위해서다.
3-6	`read_fastq` 함수 정의. 파일을 열어 `readlines()` 로 **모든 줄을 한꺼번에** 리스트로 읽는다.
8	0, 4, 8, … 처럼 4줄씩 건너뛰며 각 read 의 첫 줄 위치 `i` 를 만든다.
9	`rstrip()` 으로 줄 끝 개행을 지우고 `[1:]` 로 맨 앞 '@' 를 제거해 이름을 얻는다. 공백 뒤 설명(`1 length=101`)까지 이름에 포함된다.
10-11	`i+1` 번째 줄은 서열, `i+3` 번째 줄은 품질. `i+2`('+' 줄)는 사용하지 않는다.
12	(이름, 서열, 품질) 튜플을 `records` 리스트에 추가한다.
16	첫 번째 명령줄 인자를 파일 이름으로 함수에 넘긴다.
17-20	read 개수와 첫 read(`reads[0]`)의 이름(`[0]`), 서열(`[1]`), 서열 길이를 출력한다.

6. 반환 자료 구조

`read_fastq()` 는 튜플의 **리스트**를 돌려준다. 순서가 파일 순서대로 유지된다.

```
[
  ('SRR1553607.1 1 length=101', 'GTTAGCG...', 'BB@FFFF...'),
  ('SRR1553607.2 2 length=101', 'GGTGTAA...', '?@;BDDD...'),
  ...
]
```

다른 스크립트에서 함수만 가져다 쓰고 싶다면 주의할 점이 있다. 16-20번째 줄이 `if __name__ == "__main__":` 블록 안에 있지 않으므로, `from parse_fastq import read_fastq` 처럼 import 하면 그 순간 `sys.argv[1]` 을 읽으려다 오류가 날 수 있다.

7. 주의 사항 및 한계

- **메모리**: 파일 전체를 메모리에 올린다. 56 MB 파일은 문제없지만(약 1초), 수 GB 크기의 FASTQ 에는 적합하지 않다.

- **엄격한 4줄 형식 가정**: 서열·품질이 여러 줄로 나뉜 FASTQ, 중간에 빈 줄이 있는 파일, 줄 수가 4의 배수가 아닌 잘린 파일은 잘못 읽히거나 `IndexError` 가 난다. '@'·'+' 형식 검사도 하지 않는다.
- **압축 파일 미지원**: `.fastq.gz` 는 직접 읽을 수 없다. 먼저 `gunzip` 으로 풀어서 사용한다.
- **인자 누락·빈 파일**: 인자 없이 실행하거나 빈 파일을 주면 `IndexError: list index out of range` 로 종료된다.
- **품질 값**: 품질 문자열은 저장만 하고 출력·계산에는 쓰지 않는다. Phred 점수가 필요하면 `ord(ch) - 33` 으로 변환한다.
- **paired-end**: R1, R2 파일을 각각 따로 실행해야 한다. 두 파일의 read 순서·개수는 같다.

8. 함께 쓰면 좋은 명령어

```
# read 개수 빠르게 확인 (줄 수 ÷ 4)
echo $(( $(wc -l < data/SRR1553607_1.fastq) / 4 ))

# 앞 2개 read(8줄) 미리 보기
head -8 data/SRR1553607_1.fastq
```